

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: PROYEC-20 — Validador de Prompts y Tokens

1. CONTEXTO

El equipo de Ingeniería SDET requiere una utilidad interna de baja fricción para estimar tokens y latencia de prompts antes de integrarlos al orquestador principal. El requerimiento impone explícitamente una restricción de arquitectura Single-File: máximo 2 archivos en total. No hay usuarios concurrentes masivos, no hay persistencia de datos, no hay autenticación, no hay estado compartido entre sesiones. Es una herramienta de desarrollo local.

2. DECISIÓN ARQUITECTÓNICA

Arquitectura elegida: Static Frontend + Micro-Backend Local (2 archivos, sin dependencias de infraestructura externa)

- `index.html` !' Todo el cliente: HTML5 semántico + Tailwind CSS vía CDN + JavaScript vanilla con Fetch API nativa.
- `main.py` !' Servidor FastAPI con un único endpoint POST `/api/analyze`, CORS configurado para localhost, lógica de estimación matemática de tokens y simulación de latencia con `asyncio.sleep()`.

Stack tecnológico final:

- Frontend: HTML + Tailwind CDN + Vanilla JS (CERO frameworks JS — React/Vue/Svelte son YAGNI absoluto aquí)
- Backend: FastAPI + Uvicorn (servidor ASGI estándar de Python)
- Infraestructura externa: **NINGUNA** (0 bases de datos, 0 colas, 0 cachés)

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS

- **¿Por qué NO microservicios?** Hay un único dominio funcional (analizar texto), un único equipo consumidor (SDET local), y cero requisito de despliegue independiente. Microservicios añadirían orquestación, service discovery y overhead de red completamente injustificados.
- **¿Por qué NO base de datos?** No existe criterio de aceptación que requiera persistir históricos de análisis. Agregar PostgreSQL o SQLite violaría YAGNI y excedería el límite de 2 archivos.
- **¿Por qué NO React/Vue?** La UI tiene exactamente 3 elementos interactivos (textarea, botón, panel de resultados). Un framework JS añadiría un bundle de >100KB, un proceso de build y complejidad de estado innecesaria. Vanilla JS resuelve el problema en <50 líneas.
- **¿Por qué NO RabbitMQ/Redis para la simulación de latencia?** `asyncio.sleep()` con un valor aleatorio entre 0.5 y 2.0 segundos es matemáticamente equivalente para el propósito de simulación. Agregar un broker de mensajes sería ingeniería de ficción.
- **¿Por qué NO un bundler (Webpack/Vite)?** El requerimiento especifica Tailwind vía CDN explícitamente. Un proceso de build contradice el objetivo de 'baja fricción' y viola la restricción de 2 archivos.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (Bajo):** Tailwind CDN no es apto para producción (carga el CSS completo ~3MB). Aceptado: esta es una herramienta interna de desarrollo, nunca se desplegará en producción.
- **Riesgo 2 (Bajo):** La estimación de tokens ('palabras * 1.3') es una aproximación heurística, no un tokenizador real (ej. tiktoken de OpenAI). Aceptado: el criterio de aceptación especifica explícitamente 'cálculo matemático simple'.
- **Riesgo 3 (Muy Bajo):** Sin HTTPS, el CORS de localhost es permisivo. Aceptado: herramienta local, sin datos sensibles, sin autenticación requerida.

- **Riesgo 4 (Bajo):** Si en el futuro se requiere historial de análisis o autenticación, este diseño deberá refactorizarse. Decisión consciente: no se anticipa esa necesidad hoy (Anti-BDUF).

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 22,186
Tokens de salida (output): 7,731
Total tokens: 29,917
Horas-hombre estimadas: ~2.7 horas
Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: 2,147 input | 1,237 output | 1 llamadas
MICROPLANIFICACION-DISEÑO: 4,458 input | 2,998 output | 2 llamadas
FRONTEND-DESARROLLO: 6,552 input | 2,439 output | 2 llamadas
QA-VALIDACION: 9,029 input | 1,057 output | 2 llamadas